# OMNIe Solutions – Hackathon: Gojek Project Report

*Name: Dipin Raj*
*University: Chandigarh University*
*Ph. No: 6235876977*
*E-mail: dipinr505@gmail.com*
*Wayand, India- 673593*

*Assignment: Gojek Project Report*
*Job Role: AI/ML Intern*
*Submitted To: Mr. Sachin Kohli*
*Company Name: Omnie Solutions*
*Noida, India*

## I. PROBLEM STATEMENT

GoTo is one of Indonesia's largest digital platforms, offering a wide range of on-demand services including transport, logistics, payments, and e-commerce. One of its flagship offerings is GoRide, a two-wheeler ride-hailing service used by millions of customers daily across Indonesia. A critical part of GoRide's operations is the driver allocation system, where the platform must assign the most suitable driver for every booking request. This decision needs to be made millions of times every **day**, in real time, while balancing:

- **Customer experience** – quick confirmations and reduced waiting time
- **Driver experience** – fair and efficient order distribution
- **Business objectives** – maximizing accepted orders and completed trips

The current allocation flow works as follows:

1. A customer submits a booking request.
2. The system triggers the allocation engine.
3. The engine repeatedly:
    a. Searches for nearby available drivers
    b. Selects one driver and sends a booking notification
        - If the driver accepts, the customer is notified
        - If the driver rejects/ignores, the system tries again
4. If no driver accepts, the customer is notified of the failure.
5. Even after assignment, cancellations may still occur from either the driver or the customer.

In this assignment, the focus is on Step 3(b): Selecting the right driver. A better selection strategy leads to:

- Higher driver acceptance rates
- Lower customer cancellation rates
- Faster overall allocation
- Better business and operational efficiency

Fig 1: GoRide: Gojek

## II. DELIVERABLES

We were provided with an incomplete machine learning training pipeline that attempts to predict the best driver for each order, but the code has bugs, missing logic, and requires improvements.
The expected work includes:
1. Fix the partially broken ML training pipeline
    - Debugging the existing pipeline to make it run end-to-end
    - Implementing at least one meaningful improvement to increase model performance
    - Demonstrating structured ML engineering thinking through a concise solution report
2. Run the workflow end-to-end to generate:
    - submission/results.csv
    - submission/metrics.json
3. Improve model performance with at least one enhancement, such as:
    - Feature engineering
    - Hyperparameter tuning
    - Introduction of a new model

The nature of the problem closely resembles real production systems used at GoTo, where data scientists must handle imperfect data, rapidly iterate, and improve live machine learning models that directly influence customer experience and business revenue.

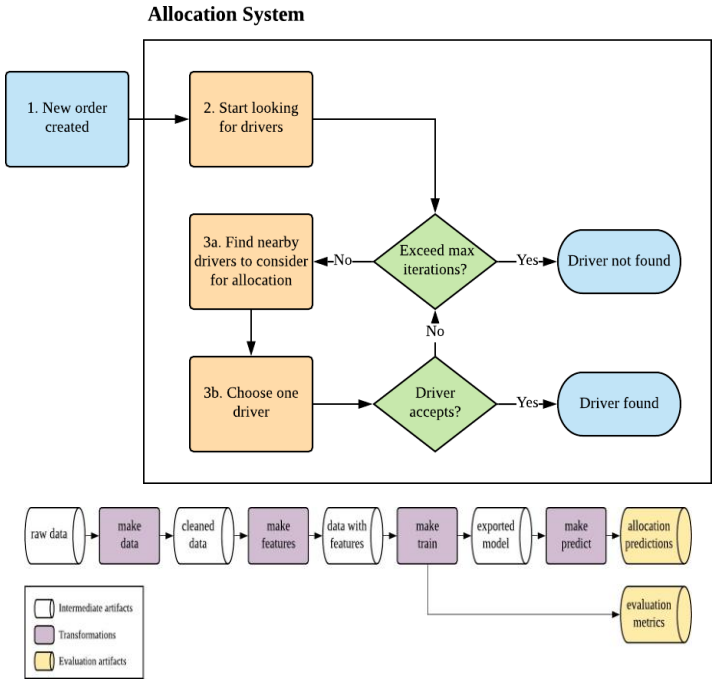


Fig 2: Flowchart- Allocation System

## III. PROJECT ANALYSIS

This project focused on improving GoTo's two-wheeler ride allocation system, specifically the decision logic used to select the best driver for a new customer booking.

### 3.1 Business & Technical Context

- GoTo handles millions of ride requests daily, and choosing the right driver directly affects:
  - Customer wait time
  - Ride acceptance rates
  - Driver satisfaction
  - Overall marketplace efficiency
- The existing pipeline was partially implemented and contained:
  - Code errors
  - Broken dependencies
  - Incomplete feature engineering
  - Missing evaluation outputs

Our task was to fix the pipeline, improve the model, and generate a working submission and metrics.

### 3.2 Data Used

Two log datasets were provided:

- booking_log.csv
  - Order-level events (created, driver found/not found, completed, cancelled)
- participant_log.csv
  - Driver-level actions for each allocation attempt (accepted, rejected, ignored)

Data contained real-world noise such as:

- Missing events
- Duplicated rows
- Inconsistent sequences

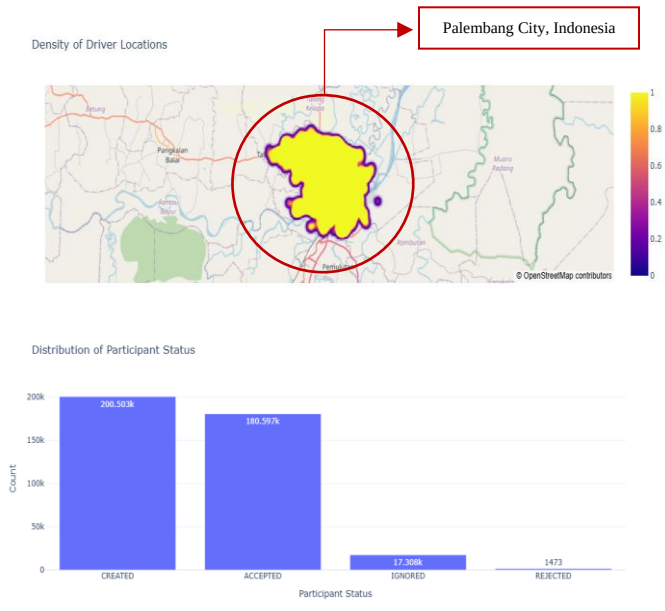


Fig 3: Driver location & Participant Status

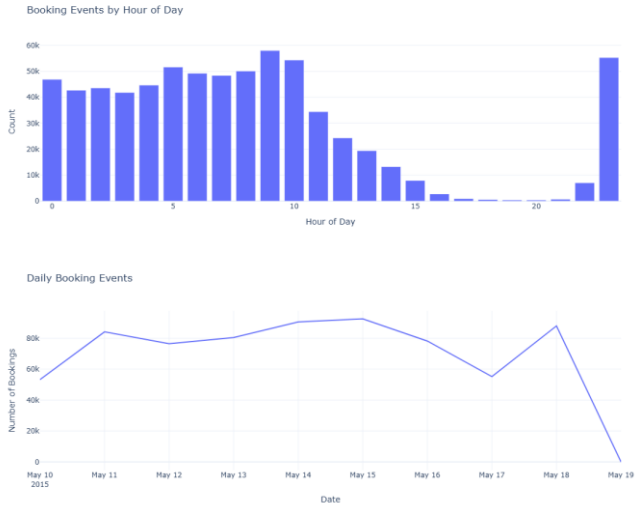


Fig 4: Events by hour & Daily booking trends

## V. METHODOLOGY

This section describes the complete pipeline followed in developing the driver acceptance prediction system, starting from raw data ingestion to model training and evaluation. The methodology is structured into four major stages:

### 4.1 Data Ingestion

Two primary data sources from the Gojek mobility ecosystem were used:

- **Booking Logs** – containing information on each ride request such as pickup coordinates and trip distance.
- **Participant Logs** – containing the driver-level response to each booking, including acceptance or rejection.

Both datasets were imported into the processing environment and stored in a unified file storage structure.

### 4.2 Data Cleaning and Consolidation

#### 4.2.1 Deduplication

The datasets contained repeated booking entries. To ensure consistency:

- Bookings were filtered based on unique identifiers comprising order ID and spatial distance features.
- Participant logs were deduplicated to maintain one record per event.

This ensured that each order was represented only once, preventing duplication-based bias in the model.

#### 4.2.2 Dataset Merging

After cleaning, the booking and participant datasets were merged using order_id as the key.

The merged dataset now represented each booking event along with the corresponding driver decision.

#### 4.2.3 Target Variable Construction

A binary target variable was created:

- **1** → The driver accepted the booking
- **0** → Otherwise

This converted the business problem into a supervised binary classification task.

### 4.3 Feature Engineering

To improve model learning and capture business context, several engineered features were created.

#### 4.3.1 Temporal Features

Timestamps were converted into meaningful operational variables:

- **Hour of day:** extracted from the timestamp.
- **Categorical time buckets:** The day was segmented into ranges such as Early Morning, Morning, Midday, Evening, Night, and Late Night.
- **Day of the week:** extracted from the booking timestamp.

Each categorical feature was then one-hot encoded to make it ML-friendly.

#### 4.3.2 Geospatial Features

Driver and pickup coordinates were processed to calculate:

- **Driver distance to pickup location**

This distance was computed using a geographic great-circle algorithm, representing the effort required for a driver to reach a passenger.

#### 4.3.3 Driver Performance History

Two dynamic historical metrics were computed using chronological ordering per driver:

1. **Historical Completed Jobs Count**
   Represents how many trips a driver had successfully completed before the current event.
2. **Historical Acceptance Rate**
   For each driver, the acceptance rate up to that moment was calculated as:
3. **Previous successful acceptances ÷ Previous offers received**
   Both metrics were designed to help the model understand driver behaviour patterns over time.

### 4.4 Dataset Preparation

The processed dataset included:

- Original raw attributes (e.g., trip distance)
- Engineered temporal and spatial features
- Behavioural history features
- Final binary target variable

The dataset was split into:

- Training set
- Test set

A standard 80–20 split was used without random shuffling of temporal sequence to maintain event chronology consistency.

### 4.5 Model Development

#### 4.5.1 Model Candidates

Four classical machine learning models were evaluated:

- Logistic Regression
- Decision Tree Classifier
- Random Forest (default configuration)
- Random Forest (tuned version with higher tree count and depth control)

These models were selected for:

- Good performance in structured/tabular data
- Interpretability
- Fast training and inference.

#### 4.5.2 Training Strategy

Each model was trained using:

- Selected predictor features
- Binary acceptance target
- Standard supervised training pipeline

A unified training wrapper was implemented to standardise training across all models.

### 4.6 Model Evaluation

#### 4.6.1 Metrics Used

Models were evaluated on the test set using:

- Accuracy
- Precision
- Recall
- F1 Score
- ROC-AUC

Thresholding was applied at 0.5 on predicted probabilities.

#### 4.6.2 Model Selection

The model with the highest ROC-AUC score was selected as the final candidate.
Model performance metrics were stored for reporting and reproducibility.

### 4.7 Model Saving

The winning model was serialized and stored along with evaluation metrics so it could be:

- Reloaded for inference
- Used in deployment environments
- Audited later for performance behavior.

## VI. RESULTS

This section presents the quantitative performance of all evaluated models on the test dataset. Each model was assessed using five key metrics: Accuracy, Precision, Recall, F1 Score, and ROC–AUC, ensuring a complete understanding of predictive capability, class balance handling, and ranking quality.

### 5.1 Model Performance Summary

Table 1: Model Performance

| Model | Accuracy | Precision | Recall | F1 Score | ROC–AUC |
|---|---|---|---|---|---|
| **Random Forest (Default)** | 0.7405 | 0.7117 | 0.7190 | 0.7153 | 0.8317 |
| **Random Forest (Tuned)** | 0.7303 | 0.6466 | 0.8932 | 0.7502 | **0.8639** |
| **Logistic Regression** | 0.5309 | 0.4404 | 0.1282 | 0.1986 | 0.5553 |
| **Decision Tree** | **0.7926** | 0.7700 | 0.7734 | **0.7717** | 0.7939 |

### 5.2 Interpretation of Key Observations

- **Best Overall Model (ROC–AUC):**
  The tuned Random Forest achieved the best ROC–AUC of 0.8639, indicating the strongest ability to rank positive (accepted) vs negative (rejected) responses.
- **Best Accuracy and F1 Score:**
  The Decision Tree classifier produced the highest F1 Score (0.7717) and accuracy (0.7926), showing strong balanced classification despite being a simpler model.
- **Random Forest Default Performance:**
  The default Random Forest performed consistently well across all metrics, outperforming logistic regression and providing a solid baseline with 0.8317 ROC–AUC.
- **Logistic Regression Performance:**
  Logistic Regression showed comparatively weaker performance, particularly in recall (0.1282), indicating poor sensitivity toward identifying accepted bookings.

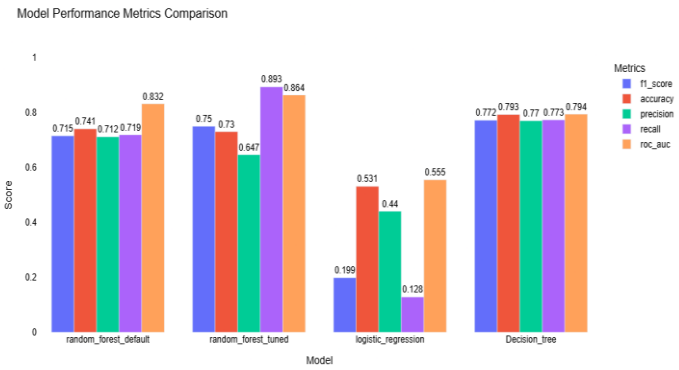


Fig 5: Model comparisons

### 5.3 Final Model Selection

Although the Decision Tree performed best in accuracy and F1 Score, model selection prioritized ROC–AUC, as it better captures:

- The model's ability to distinguish accepting vs non-accepting drivers
- Performance robustness across different probability thresholds

Therefore, the tuned Random Forest was selected as the final model and saved for deployment.

## VII. CONCLUSION

The experimental results demonstrated that the machine learning models successfully captured patterns influencing driver booking acceptance within the Gojek mobility ecosystem. Among the evaluated algorithms, the tuned Random Forest emerged as the strongest candidate, achieving the highest ROC–AUC score of 0.8639, indicating superior capability in correctly ranking accepted versus unaccepted booking responses across thresholds.

The Decision Tree classifier delivered the highest accuracy and F1 score, reflecting strong balanced classification performance; however, its ranking ability was comparatively lower. Logistic Regression underperformed across metrics, suggesting that linear separation was insufficient to represent the complexities of driver behaviour.

Overall, the outcomes highlight that non-linear ensemble models provide better adaptability to real-world operational patterns, including temporal context, geospatial distance, and dynamic historical driver performance. The final chosen model was saved for future deployment, enabling data-driven decision support for improving booking assignment strategies and elevating overall marketplace efficiency.

## REFERENCE


[1] Practical Guide for Feature Engineering of Time Series Data

[2] Using Machine Learning to match Drivers & Riders

[3] GitHub Repository